

作业答案-类和对象

1. 从概念上讲，抽象与封装有什么不同？

答：处理大而复杂问题的重要手段是抽象：强调事物本质的东西。对程序抽象而言，一个语言结构的抽象强调的是该结构外部可观察到的行为，与该结构的内部实现无关。抽象包括过程抽象(Procedural Abstraction)和数据抽象(Data Abstraction)。

封装是把一个语言结构的具体实现细节作为一个黑匣子对该结构的使用者隐藏起来的一种机制，从而符合信息隐藏(Information Hiding)原则。封装包括过程封装和数据封装。

封装考虑的是内部实现，抽象考虑的是外部行为。

2. 对于一个类定义，哪些部分应放在头文件（.h）中，哪些部分放在实现文件（.cpp）中？

答：一般情况下，类定义放在头文件（.h）中，类外定义的成员函数放在实现文件（.cpp）中。

3. 对类成员的访问，C++提供了哪些访问控制？

答：在 C++ 的类定义中，可以用访问控制修饰符 `public`, `private` 和 `protected` 对类成员的访问进行限制。`public`：访问不受限制。`private`：只能在本类和友元中访问。`protected`：只能在本类、派生类和友元中访问。

4. 在 C++ 中，`this` 指针的作用是什么？

答：C++ 中类定义中声明的非静态数据成员对该类的每个对象都有一个拷贝，而成员函数是被该类的每个对象共享的，为了确定成员函数中用到的数据成员是属于哪一个对象的，C++ 采用了一个 `this` 指针解决这个问题：每个成员函数都有一个缺省的形式参数 `this`，其类型为指向该类对象的指针；调用一个对象的成员函数时，实现系统会把该对象的地址传给成员函数。在成员函数中访问类中定义的数据成员时，实际访问的是 `this` 所指向的对象的数据成员。

5. 构造函数成员初始化表可以包含哪些内容？

答：在定义构造函数时，函数头和函数体之间可以加入一个对数据成员进行初始化的表，用于对数据成员进行初始化，特别是对常量和引用数据成员进行初始化。如需要调用成员对象和基类的非默认构造函数，也需要在对象类构造函数的成员初始化表中指定。

6. 拷贝构造函数的作用是什么？何时会调用拷贝构造函数？

答：拷贝构造函数用于在创建对象时用另一个同类的对象对其初始化。一般来说，有三种情况将调用拷贝构造函数：

- 1) 定义对象时
- 2) 把对象作为值参数传给函数时

3) 把对象作为返回值时

7. 对于一个类 A，为什么下面的构造函数不合法？

```
A(A x);
```

答：因为，对于下面的程序将会不停地递归调用 A(A x)：

```
A a;
```

```
A b(a); //调用 b.A(a)时，将会调用 x.A(a)，a.A(x)，...，从而形成无穷递归！
```

8. 在创建包含成员对象的类的对象时，为什么要首先初始化成员对象？

答：因为在包含成员对象的构造函数中可能用到成员对象，如果这时成员对象未初始化，则会用到未初始化的对象！例如：

```
class A
{
    ...
    void f();
};
class B
{
    A a;
public:
    B()
    { a.f(); //如果这时a未初始化，则a.f() 没有意义。
    }
};
B b; //调用b的构造函数。
```

9. 何时需要定义析构函数？

答：如果对象在创建后申请了一些资源并且没有归还这些资源，则应定义析构函数来在对象消亡时归还对象申请的资源。

10. 静态数据成员的作用是什么？静态数据成员如何初始化？

答：在 C++中采用静态成员来解决同一个类的对象共享数据的问题。类定义中的静态数据成员对该类的所有对象只有一个拷贝，即它们被该类的所有对象所共享。另外，虽然全局变量也能起到共享的效果，但静态数据成员较为安全，因为静态数据成员还可以受到类的访问控制的保护。

静态数据成员必须要在类的外部给出它们的定义，定义时可以进行初始化。

11. 类作用域与局部作用域有什么不同？

答：当标识符的声明出现在由一对花括号所括起来的一段程序内（块，block）时，该（局部）标识符的作用域从声明点开始，到块结束处为止，该作用域的范围具有局部性。而类作用域是指类定义和相应的成员函数定义范围。在该范围内，一个类的成员函数对同一类的数据成员具有无限制访问权。

12. 在定义一个包含成员对象的类时，表示成员对象的数据成员何时定义为成员对象指针和成员对象本身？

答：当一个对象 b（类为 B）作为另一个对象 a（类为 A）的组成部分，并且组成关系不会发生变化（一个学生与他的姓名，出生日期之间的关系），则应把 b 定义为 A 类的成员对象。当对象 b 作为另一个对象 a 的组成部分，但组成关系会发生变化（如项目组和项目成员的组成关系），或者对象 b 不是 a 的一部分，但它们之间有关联（部门经理和成员之间的管理与被管理关系），这时应把 b 定义为 A 类的成员对象指针。

13. 为什么要对操作符进行重载？是否所有的操作符都可以重载？

答：通过对 C++ 操作符进行重载，我们可以实现用 C++ 的操作符按照通常的习惯来对某些类（特别是一些数学类）的对象进行操作，从而使得程序更容易理解。除此之外，操作符重载机制也提高了 C++ 语言的灵活性和可扩充性，它使得 C++ 操作符除了能对基本数据类型和构造数据类型进行操作外，也能用它们来对类的对象进行操作。

不是所有的操作符都可以重载，因为 “.”， “.*”， “::”， “?:”， sizeof 这五个操作符不能重载。

14. 操作符重载的形式有哪两种形式？这两种形式有什么区别？

答：一种就是作为成员函数重载操作符；另一种就是作为全局（友元）函数重载操作符。

当操作符作为类的非静态成员函数来重载时，由于成员函数已经有一个隐藏的参数 this，因此对于双目操作符重载函数只需要提供一个参数，对于单目操作符重载函数则不需提供参数。

当操作符作为全局函数来重载时，操作符重载函数的参数类型至少有一个为类、结构、枚举或它们的引用类型。而且如果要访问参数类的私有成员，还需要把该函数说明成相应类的友元。对于双目操作符重载函数需要两个参数，对于单目操作符重载函数则需要给出一个参数。操作符=、()、[]以及->不能作为全局函数来重载。

另外，作为类成员函数来重载时，操作符的第一个操作数必须是类的对象，全局函数重载则否。

15. 定义一个类 A，使得在程序中只能创建一个该类的对象，当试图创建该类的第二个对象时，返回第一个对象的指针。

答：

```
//Design of class Singleton:
class Singleton
{ public:
    static Singleton* Instance();
protected:
    Singleton();
private:
    static Singleton* _instance;
}

Singleton* Singleton::_instance = 0;

Singleton* Singleton::Instance()
{ if (_instance == 0)
    { _instance = new Singleton;
    }
    return _instance;
};
```

```
//How to Use Singleton:  
Singleton* singleton = Singleton::Instance();
```